Linear Operators, Images, and Convolution

Mathematics of Deep Learning — Chapter 4

Yin Yang

Foundation Books Project

Roadmap

Pixels and Neighborhoods

Images as Tensors

Dense Maps on Flattened Images

Sliding Dot Products

Cross-Correlation vs. Convolution

Convolution as a Linear Operator

Padding, Stride, Dilation

Channels and Filter Banks

Parameter Sharing and Equivariance

Pooling and a Minimal CNN

Summary

Pixels and Neighborhoods

From Numbers Back to Geometry

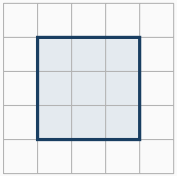
Earlier chapters turned digit images into arrays. This chapter looks at the structure those arrays already carry.

- One image: $X \in [0,1]^{28\times 28}$.
- Row and column indices identify *neighboring* pixels.
- A stroke is not one pixel — it is a local pattern across several nearby pixels.

Question of the chapter: how can a linear map use *neighbor relations* among pixel values, without learning a separate rule for every coordinate?

Flattening keeps the values. Spatial axes keep the neighbor relation visible.

Local Patches as the Basic Unit



local 3×3 patch

flattened vector

- A spatial grid marks which pixels are neighbors.
- After flattening, the same values remain.
- The vector shape no longer marks the neighborhood — a dense layer cannot tell which weights are "adjacent".

Convolution is the linear map that respects the grid.

Images as Tensors

Channel-Last Image Layout

Convention used throughout this chapter:

$$\text{one image: } h \times w \times c, \qquad \text{batch of } m \text{ images: } m \times h \times w \times c.$$

Object	Shape	Meaning
one grayscale digit	$28 \times 28 \times 1$	H, W, intensity
batch of grayscale digits	$m \times 28 \times 28 \times 1$	batch + image axes
one RGB image	$h \times w \times 3$	R, G, B channels
batch of RGB images	$m \times h \times w \times 3$	batch + color axes

The channel axis stores different measurements *at the same location*. It is not a second batch axis.

Mixing the batch axis with the channel axis changes the object being fed to the model.

What the Channel Axis Really Indexes

For an RGB image,

$$X_{i,j,1}, \quad X_{i,j,2}, \quad X_{i,j,3}$$

are red, green, and blue measurements at row i, column j. For a learned feature map,

$$X_{i,j,c} = \text{response of detector } c \text{ at location } (i, j).$$

Concrete batch: $32 \times 28 \times 28 \times 1$.

- $X_{7,12,20,1}$ = intensity, example 7, row 12, column 20.
- A conv layer operates on the last three axes per example.
- The same kernel weights are shared across the batch.

Stored as $m \times c \times h \times w$? Permute first — or a kernel will slide along the wrong axis.

Dense Maps on Flattened Images

Dense Layer on a Flattened Image

Flatten the image and apply an affine layer:

$$x = \mathrm{vec}(X) \in \mathbb{R}^{784}, \qquad z = Wx + b, \quad W \in \mathbb{R}^{q \times 784}.$$

Parameter counts on the 28×28 digit:

Layer	Trainable parameters
flattened image $\to$ 64 hidden features	$64 \cdot 784 + 64 = 50{,}240$
flattened image $\to$ 10 class scores	$10 \cdot 784 + 10 = 7{,}850$

Manageable here. Not manageable for larger images.

A dense layer connects every pixel to every output — but it does not encode which pixels were neighbors.

Receptive Field and Reused Detectors

Receptive field
For one output coordinate of a layer, the set of input coordinates that can affect it.

In a dense layer on a flattened image, the receptive field of every output is the *entire* image.

- A small vertical stroke may appear left, center, or right.
- Dense weights at each location are unrelated.
- Training must learn the same local detector *many times*, once per position.

Convolution reuses small local weights across locations instead of relearning them.

Sliding Dot Products

From Dot Product to Sliding Detector

Local patch and kernel of the same shape:

$$P_{i,j} = (X_{i+u,j+v})_{0 \le u < k_h,\, 0 \le v < k_w}, \qquad K \in \mathbb{R}^{k_h \times k_w}.$$

One number per patch:

$$\langle K, P_{i,j} \rangle = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v}\, X_{i+u,j+v}.$$

Sliding K over all valid (i,j) produces a *spatial feature map*.

- One kernel = one local detector.
- The output at (i,j) depends only on a small index set.
- All other inputs have coefficient zero in that output.

Locality is enforced by which input coordinates are even allowed into the sum.

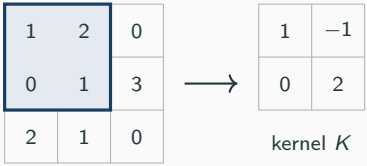
A Hand Calculation, Step by Step

1	2	0
0	1	3
2	1	0

image X

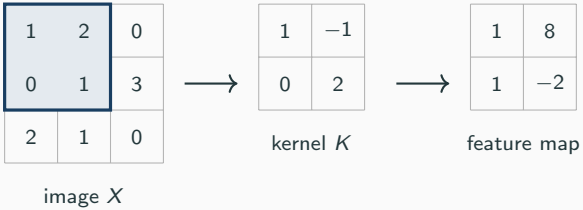
(1) Image and the upper-left 2×2 patch.

A Hand Calculation, Step by Step



(1) Image and the upper-left 2×2 patch.
(2) Kernel K has the same spatial shape.

A Hand Calculation, Step by Step



(1) Image and the upper-left 2×2 patch.
(2) Kernel K has the same spatial shape.
(3) Upper-left response: $1 \cdot 1 + (-1) \cdot 2 + 0 \cdot 0 + 2 \cdot 1 = 1$.

Same kernel, same arithmetic, repeated at every valid patch position.

Cross-Correlation vs. Convolution

Two Formulas, One Architecture

Valid cross-correlation

$$\operatorname{corr}(X, K)_{i,j} = \sum_{u,v} K_{u,v}\, X_{i+u, j+v}.$$

Valid mathematical convolution

$$\operatorname{conv}(X, K)_{i,j} = \sum_{u,v} K_{u,v}\, X_{i+k_h-1-u,\, j+k_w-1-v}.$$

Equivalence via the 180°-rotated kernel $\widetilde{K}$:

$$\operatorname{conv}(X, K) = \operatorname{corr}(X, \widetilde{K}).$$

Most deep-learning libraries compute cross-correlation but still call the layer “convolutional”.

The Kernel Flip on One Patch

For $K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$, the rotated kernel is $\widetilde{K} = \begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}$. On the upper-left patch $\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$:

Operation	Formula at $(0,0)$	Value
correlation, K	$1\cdot1 + (-1)\cdot2 + 0\cdot0 + 2\cdot1$	1
true convolution, K	$2\cdot1 + 0\cdot2 + (-1)\cdot0 + 1\cdot1$	3

When the kernel is learned, the flip does not change the function family — training can absorb the rotation. The flip matters when comparing to a hand calculation or a fixed signal-processing filter.

Different conventions, same patch, different numbers — check the convention before declaring a bug.

Convolution as a Linear Operator

The 1D Convolution Matrix

Kernel $k = (a, b, c)$, input length $n = 5$:

$$C_k = \begin{bmatrix} a & b & c & 0 & 0 \\ 0 & a & b & c & 0 \\ 0 & 0 & a & b & c \end{bmatrix}, \qquad C_k x = \begin{bmatrix} ax_1 + bx_2 + cx_3 \\ ax_2 + bx_3 + cx_4 \\ ax_3 + bx_4 + cx_5 \end{bmatrix}.$$

- Dense 3×5 matrix: 15 independent weights.
- Convolution matrix: only 3.
- Same a, b, c shifted one position per row — a *Toeplitz* pattern.

Convolution is a constrained linear map: sparse local connections plus repeated weights.

Linearity and the 2D Sparse Matrix

For a fixed kernel K, $X \mapsto \mathrm{corr}(X, K)$ is linear in X:

$$\mathrm{corr}(\alpha X + \beta Z, K) = \alpha\ \mathrm{corr}(X, K) + \beta\ \mathrm{corr}(Z, K).$$

Matrix representation
Flattening row by row, there exists

$$C_K \in \mathbb{R}^{(h-k_h+1)(w-k_w+1)\times hw}$$

with $\mathrm{vec}(\mathrm{corr}(X, K)) = C_K\,\mathrm{vec}(X)$.

Each row of C_K has nonzeros only on one local patch, and those nonzeros are the kernel weights. In 2D the pattern is sparse block-Toeplitz with Toeplitz blocks.

Before the activation, a conv layer is multiplication by a sparse structured matrix.

Padding, Stride, Dilation

Three Shape Knobs

Padding p
Add values (typically zeros) around the boundary before applying the kernel.

Stride s
Move the sliding window by s positions between neighboring outputs (subsample the starts).

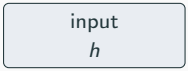
Dilation d
Space out the entries of the kernel. A 3×3 kernel with $d = 2$ covers a 5×5 span using 9 weights.

Effective kernel size:

$$k_{\text{eff}} = d(k-1) + 1.$$

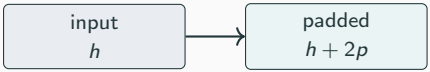
Mistakes in p, s, d change the tensors passed to every later layer.

The Output-Shape Formula, Built Up



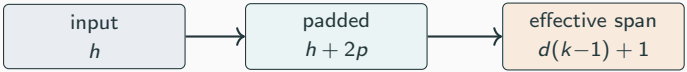
(1) Start with input height h.

The Output-Shape Formula, Built Up



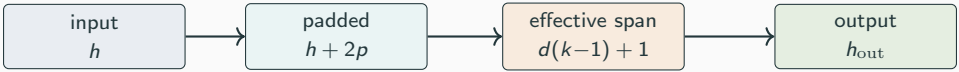
(1) Start with input height h.
(2) Padding extends to effective length $h + 2p$.

The Output-Shape Formula, Built Up



(1) Start with input height h.
(2) Padding extends to effective length $h + 2p$.
(3) The last start must fit a span of $d(k-1) + 1$.

The Output-Shape Formula, Built Up



(1) Start with input height h.

(2) Padding extends to effective length $h + 2p$.

(3) The last start must fit a span of $d(k-1)+1$.

(4) Starts sampled every s: $h_{\text{out}} = \left\lfloor \dfrac{h + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$

Width formula is identical with the width parameters.

Shape Table for the Running Digit

Input is 28×28.

Kernel	Padding	Stride	Dilation	Output
3×3	0	1	1	26×26
3×3	1	1	1	28×28
3×3	1	2	1	14×14
5×5	0	1	1	24×24
3×3	2	1	2	28×28

Stride 2 halves the grid. Padding 1 with a 3×3 kernel preserves resolution.

The shape table is the first check before a layer is even built.

Channels and Filter Banks

Multi-Channel Filter Bank

Input $X \in \mathbb{R}^{h \times w \times c_{\text{in}}}$, weights and bias

$$W \in \mathbb{R}^{k_h \times k_w \times c_{\text{in}} \times c_{\text{out}}}, \qquad b \in \mathbb{R}^{c_{\text{out}}}.$$

Output channel r:

$$Y_{i,j,r} = b_r + \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \sum_{c=1}^{c_{\text{in}}} W_{u,v,c,r}\, X_{i+u,j+v,c}.$$

- Sums over u, v: walk a local spatial patch.
- Sum over c: combine all input channels at that location.
- Index r: separate learned detectors stacked along the output channel axis.

Each output channel has its own filter, and each filter spans every input channel.

Parameter Counts: Conv vs. Dense

Parameter count of a multi-channel conv layer:

$$k_h \, k_w \, c_{\text{in}} \, c_{\text{out}} + c_{\text{out}}.$$

For RGB input ($c_{\text{in}} = 3$) and 5×5 kernel with $c_{\text{out}} = 12$:

Layer	Parameters
5×5 conv, $3 \to 12$ channels	$5 \cdot 5 \cdot 3 \cdot 12 + 12 = 912$
dense $16{\cdot}16{\cdot}3 \to 12$ on flattened 16×16 RGB	$(16 \cdot 16 \cdot 3) \cdot 12 + 12 = 9{,}228$

The conv layer is smaller because one $5 \times 5 \times 3$ filter is *reused* at every spatial location — not because the channels disappear.

Adding one output channel adds only $k_h k_w c_{\text{in}}$ weights.

im2col: Conv as Matrix Multiplication

Stack multi-channel patches as rows:

$$P_X \in \mathbb{R}^{N_{\text{patches}} \times (k_h k_w c_{\text{in}})}.$$

Flatten each filter into a column:

$$W_{\text{col}} \in \mathbb{R}^{(k_h k_w c_{\text{in}}) \times c_{\text{out}}}.$$

Then the entire multi-channel correlation is one matrix product:

$$Y_{\text{col}} = P_X \, W_{\text{col}} + \mathbf{1} \, b^\top.$$

Same structured-linear-operator idea as the 1D Toeplitz form — each row of P_X reads one local multi-channel patch.

Conv is matrix multiplication on a patch-rearranged input.

Parameter Sharing and Equivariance

Three Properties That Make Conv Useful

Locality
The output at (i, j) sees only a small patch of the input.

Parameter sharing
The same weights $W_{u,v,c,o}$ are reused at every (i, j).

Translation equivariance
Shifting the input shifts the feature map the same way — away from boundary effects.

Equivariance means the response *moves with* the input; it does not mean the response is unchanged.

Shared Weights $\Rightarrow$ Interior Equivariance

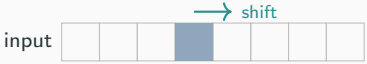
Let T_δ shift an image: $(T_\delta X)_{a,b} = X_{a+\delta_1,\, b+\delta_2}$. At any interior location where both sides are defined,

$$\begin{aligned}\operatorname{corr}(T_\delta X, K)_{i,j} &= \sum_{u,v} K_{u,v}\, (T_\delta X)_{i+u,\, j+v} \\ &= \sum_{u,v} K_{u,v}\, X_{i+\delta_1+u,\, j+\delta_2+v} \\ &= \operatorname{corr}(X, K)_{i+\delta_1,\, j+\delta_2} = (T_\delta \operatorname{corr}(X, K))_{i,j}.\end{aligned}$$

Every row of the Toeplitz matrix uses the same kernel coefficients, so a shifted patch is read by a shifted row.

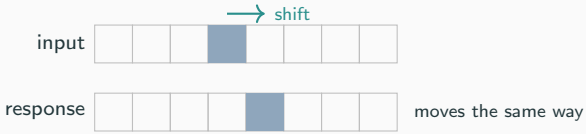
Untied dense rows would generally break this equivariance.

Boundaries Break Exact Equivariance



(1) A bright stroke at an interior location.

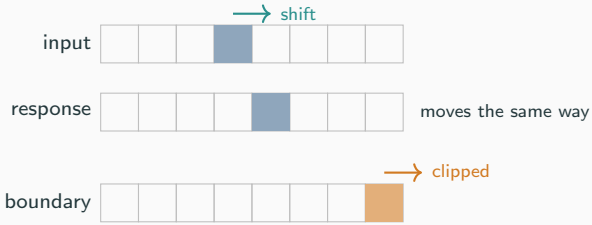
Boundaries Break Exact Equivariance



(1) A bright stroke at an interior location.

(2) Shifting the input shifts the response the same way.

Boundaries Break Exact Equivariance



(1) A bright stroke at an interior location.
(2) Shifting the input shifts the response the same way.
(3) At the boundary, padding decides which values are read — the response can be clipped or distorted.

Circular padding can make finite-grid shifts exact; zero padding depends on the exterior convention.

Equivariance, in Reverse

Two directions of the same story:

- **Forward.** Shared local weights $\Rightarrow$ interior translation equivariance (just shown).
- **Converse (informal).** On an infinite grid, or with circular boundaries, every *linear* map that commutes with all translations has convolutional form.

Add locality $\Rightarrow$ finite spatial support, i.e. a kernel. The same principle generalizes from translations to other symmetry groups via group convolution.

Linearity plus translation symmetry essentially *forces* convolutional structure.

Pooling and a Minimal CNN

Local Pooling: Fixed, Channelwise

Channelwise local pooling
Window $\mathcal{W}_{i,j}$, aggregation g:

$$P_{i,j,c} = g\big(\{X_{u,v,c} : (u,v) \in \mathcal{W}_{i,j}\}\big).$$

- $g = \max$: keep strongest local response.
- $g = \text{mean}$: smooth local responses.
- No trainable weights.
- Stride > 1: reduces spatial resolution.

Pooling trades spatial detail for smaller feature maps while keeping channels separate.

A Minimal CNN, Built Up

(1) Input digit shape.

A Minimal CNN, Built Up

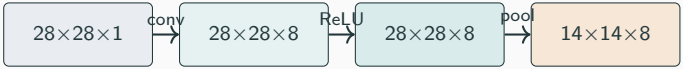
(1) Input digit shape.

(2) 3×3 conv, padding 1, 8 output channels.

A Minimal CNN, Built Up

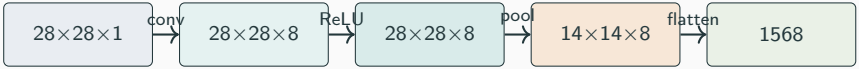
(1) Input digit shape.
(2) 3×3 conv, padding 1, 8 output channels.
(3) Elementwise ReLU — shape unchanged.

A Minimal CNN, Built Up



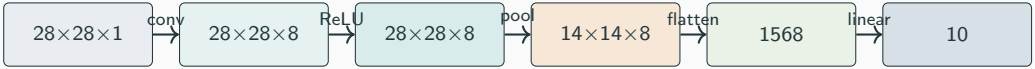
(1) Input digit shape.
(2) 3×3 conv, padding 1, 8 output channels.
(3) Elementwise ReLU — shape unchanged.
(4) 2×2 pool, stride 2: halves h, w.

A Minimal CNN, Built Up



(1) Input digit shape.
(2) 3×3 conv, padding 1, 8 output channels.
(3) Elementwise ReLU — shape unchanged.
(4) 2×2 pool, stride 2: halves h, w.
(5) Flatten the feature stack to a vector.

A Minimal CNN, Built Up



(1) Input digit shape.
(2) 3×3 conv, padding 1, 8 output channels.
(3) Elementwise ReLU — shape unchanged.
(4) 2×2 pool, stride 2: halves h, w.
(5) Flatten the feature stack to a vector.
(6) Dense classifier to ten class scores; softmax on top.

Reading a CNN is bookkeeping — which shape changes where, and where the grid is flattened.

Where the Parameters Live

Parameter count in the minimal CNN:

Layer	Parameters
3×3 conv, $1 \to 8$ channels	$3 \cdot 3 \cdot 1 \cdot 8 + 8 = 80$
ReLU, 2×2 pool, flatten	0
dense classifier $1568 \to 10$	$1568 \cdot 10 + 10 = 15{,}690$
total	15,770

The conv spends few parameters on reusable local detectors. The dense layer mixes the extracted feature grid into ten class scores — that is where most of the count lives.

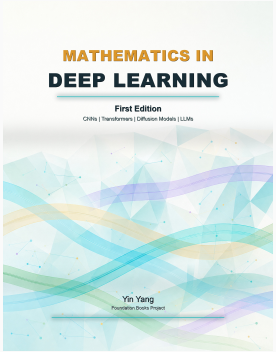
Locality and sharing are the parameter budget choice that makes CNNs scale to images.

Summary

Chapter in One Slide

- **Spatial structure.** Images carry a neighbor relation that flattening hides; convolution is the linear map that respects the grid.
- **Sliding dot products.** A kernel reads one local patch; sliding produces a spatial feature map.
- **Correlation vs. convolution.** Libraries compute correlation; the flip is a convention, not a function-family change for learned kernels.
- **Linear operator.** A fixed kernel acts as a sparse block-Toeplitz matrix on flattened pixels.
- **Shape knobs.** Padding, stride, dilation set the output shape via $\lfloor (h + 2p - d(k-1) - 1)/s \rfloor + 1$.
- **Channels.** Filter banks reuse one $k_h k_w c_{\text{in}}$-weight detector per output channel.
- **Sharing $\Rightarrow$ equivariance.** Repeated rows $\Rightarrow$ shifted inputs give shifted feature maps.
- **Minimal CNN.** conv $\to$ ReLU $\to$ pool $\to$ flatten $\to$ linear $\to$ softmax.

Next: backpropagation through convolutional, pooling, and dense layers — gradients with shared weights across many locations.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/mathematics-in-deep-learning`
Code	`github.com/foundation-books/mathematics-in-deep-learning-code`
Paperback	`amazon.com/dp/B0H1ZZHDT2`
Hardcover	`amazon.com/dp/B0H1ZL11MJ`
Kindle	`amazon.com/dp/B0GX32M8SP`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.